GenSpark

# OOP: Inheritance & Polymorphism

The last article introduced classes as a way to bundle data and behavior together. This article extends that idea. What happens when you have several related types of something that share common behavior but also need their own specific differences? Python's answer is inheritance, and the payoff is polymorphism. By the end, you will be able to build the family of AI opponents the Tic-Tac-Toe lab requires.

## The Problem Inheritance Solves

Imagine the bookstore also wants to sell eBooks alongside physical books. Both share most of the same data and behavior: a title, an author, a price, and selling a copy. But an eBook has no physical quantity to run out of, so it can be "sold" without limit, and it has a file size instead of a shelf location.

Without inheritance, you would either duplicate the entire `Book` class with small changes, or cram both cases into one class full of conditional logic checking "if this is an eBook, do X, otherwise do Y" everywhere. Inheritance offers a cleaner alternative.

## Basic Inheritance

Here is the `Book` class from the last article, with a method to describe itself:

python

```
class Book:
    def __init__(self, title, author, price, quantity=0):
        self.title = title
        self.author = author
        self.price = price
```

```
        self._quantity = quantity

    @property
    def quantity(self):
        return self._quantity

    def sell_copy(self):
        """Sell one copy. Return True if a copy was sold, False if out of
        if self._quantity > 0:
            self._quantity -= 1
            return True
        return False

    def describe(self):
        return f"{self.title} by {self.author} (${self.price:.2f})"
```

And an `EBook` that builds on it:

python

```
class EBook(Book):
    def __init__(self, title, author, price, file_size_mb):
        super().__init__(title, author, price, quantity=0)
        self.file_size_mb = file_size_mb
```

`class EBook(Book)` means `EBook` **inherits** from `Book`. It automatically gets all of `Book`'s attributes and methods without rewriting them. `super().__init__(...)` calls the parent class's `__init__` to handle the shared setup (title, author, price, quantity), so `EBook` only needs to add what is new: `file_size_mb`.

python

```
paperback = Book("Dune", "Frank Herbert", 15.50, quantity=2)
ebook = EBook("Project Hail Mary", "Andy Weir", 9.99, file_size_mb=2.8)

print(ebook.title)            # inherited from Book
print(ebook.file_size_mb)     # specific to EBook
print(ebook.describe())       # inherited method works as is
```

`Book` here is called the **parent class** (also the *base class* or *superclass*), and `EBook` is the **child class** (or *subclass*). The child gets everything the parent has, plus whatever it adds.

## Checking relationships

Two built-in functions ask about the family tree:

python

```
print(isinstance(ebook, EBook))       # True
print(isinstance(ebook, Book))        # True, an EBook IS a Book
print(isinstance(paperback, EBook))   # False, a plain Book is not an EBoo
print(issubclass(EBook, Book))        # True
print(issubclass(Book, EBook))        # False
```

`isinstance(x, SomeClass)` is `True` if `x` was built from that class *or any class that inherits from it*. The relationship only points one way: every eBook is a book, but not every book is an eBook.

Python searches for a method in a fixed order: first the object's own class, then its parent, then the parent's parent, and so on. You can see that order with `EBook.__mro__`, which lists `EBook`, `Book`, and then `object`. Every class in Python ultimately inherits from `object`, the root of the family tree.

## Don't forget super().init

If a child defines its own `__init__` and never calls the parent's, the parent's setup does not happen:

python

```
class BadEBook(Book):
    def __init__(self, title, author, price, file_size_mb):
        self.file_size_mb = file_size_mb       # forgot super().__init__(.

bad = BadEBook("Dune", "Frank Herbert", 9.99, 3.2)
bad.describe()
```

```
AttributeError: 'BadEBook' object has no attribute 'title'
```

The title, author, and price were never stored. If a child class has its own `__init__`, call `super().__init__(...)` first, passing along whatever the parent needs.

## Method Overriding

Sometimes a child class needs to change how a parent's method behaves, rather than just adding new attributes. This is called **overriding**. You do it by redefining the method in the child class with the same name.

A digital copy never runs out, so an eBook's `sell_copy()` should always succeed:

python

```
class EBook(Book):
    def __init__(self, title, author, price, file_size_mb):
        super().__init__(title, author, price, quantity=0)
        self.file_size_mb = file_size_mb

    def sell_copy(self):
        """A digital copy never runs out, so a sale always succeeds."""
        return True
```

python

```
print(paperback.sell_copy())    # True  (quantity 2 becomes 1)
print(paperback.sell_copy())    # True  (quantity 1 becomes 0)
print(paperback.sell_copy())    # False (out of stock)
print(ebook.sell_copy())        # True
print(ebook.sell_copy())        # True, no limit
```

When you call `.sell_copy()` on a regular `Book`, you get the original behavior. When you call it on an `EBook`, you get the new behavior, even though both objects respond to the exact same method name.

## Extending instead of replacing

Overriding replaces the parent's version completely. Often you want to *add to* it instead. Call the parent's version with `super()` and then do the extra part:

python

```
class EBook(Book):
    # __init__ and sell_copy as above

    def describe(self):
        return super().describe() + f", digital, {self.file_size_mb} MB"
```

python

```
print(ebook.describe())
# Project Hail Mary by Andy Weir ($9.99), digital, 2.8 MB
```

`super().describe()` runs `Book` 's version, and the child adds its own details. This avoids copying the parent's formatting code, which follows the DRY principle. If the parent's description changes later, the child picks up the change automatically.

# Polymorphism

**Polymorphism** means "many forms." Objects of different classes can be used interchangeably if they share a common interface, each responding to the same method call in its own way.

python

```
catalog = [
    Book("Dune", "Frank Herbert", 15.50, quantity=2),
    EBook("Project Hail Mary", "Andy Weir", 9.99, file_size_mb=2.8),
]

for item in catalog:
    print(item.describe())      # each object runs its own version of desc
```

```
Dune by Frank Herbert ($15.50)
Project Hail Mary by Andy Weir ($9.99), digital, 2.8 MB
```

**Use case: why this matters for the bookstore's checkout screen.** Imagine building a checkout screen that needs to display every item in a customer's cart, some physical books and some eBooks. Without polymorphism, the checkout code would need `if isinstance(item, EBook): ... else: ...` scattered everywhere it displays an item. Every time a new format gets added (audiobooks, say), that conditional logic has to be found and updated in every single place it appears. With polymorphism, the checkout code just calls `item.describe()` on whatever is in the cart, and each item type handles displaying itself correctly. Adding audiobooks later means writing one new class, not hunting down every `isinstance` check in the codebase.

## Duck typing

Python goes a step further. The loop above works for *any* object that has a `describe()` method, whether or not it inherits from `Book`:

python

```
class Audiobook:
    def describe(self):
        return "an audiobook"

for item in [paperback, ebook, Audiobook()]:
    print(item.describe())      # works for all three
```

This is called **duck typing**: "if it walks like a duck and quacks like a duck, treat it as a duck." Python does not check the family tree, only whether the method exists. Inheritance is still valuable because it shares code and signals a deliberate relationship, but polymorphism in Python does not strictly require it.

# Designing Good Hierarchies

Inheritance is easy to misuse. These guidelines keep it useful:

- **Apply the "is-a" test out loud.** "An EBook is a Book" is true, so inheritance fits. "A Game is

a Board" is false, so a `Game` should *have* a `Board` instead. Prefer composition (has-a) when in doubt.

- **A child must keep the parent's promises.** Anywhere the code expects a `Book`, an `EBook` should work. If `sell_copy()` promises to return `True` or `False`, an override must return `True` or `False`, not raise an error or return text. A child that breaks its parent's promises makes polymorphism unsafe.
- **Keep hierarchies shallow.** One or two levels is typical. A chain of five classes is hard to follow, because to understand one method you must read all of its ancestors.
- **Put shared code in the parent.** If three children repeat the same lines, those lines belong in the parent.
- **Avoid multiple inheritance** ( `class C(A, B)` ) while you are learning. Python supports it, but it creates puzzles about which parent's method wins.
- **Don't inherit just to reuse code.** If the "is-a" sentence is false, use composition or a plain helper function.

# Abstract Base Classes

Sometimes a parent class exists only to describe what its children must be able to do, and it makes no sense to create an object from it directly. Tic-Tac-Toe is a good example. There are several AI opponents, and every one of them must be able to choose a move. But "a generic AI player" that chooses nothing in particular is meaningless.

Python's `abc` module (short for *abstract base classes*) lets you say this explicitly:

python

```
from abc import ABC, abstractmethod


class AIPlayer(ABC):
    """Base class for every computer opponent."""

    @abstractmethod
    def choose_move(self, board, mark):
        """Return a (row, col) tuple for an empty cell on the board."""
```

Three pieces make this work:

- `ABC` is the base to inherit from. It marks the class as abstract.
- `@abstractmethod` is a decorator marking a method that every child *must* provide. The body here is only a docstring, because the parent has nothing to do.
- Any class that inherits from `AIPlayer` must override `choose_move`, or Python refuses to create objects from it.

You cannot create an object from the abstract class itself:

python

```
AIPlayer()
```

```
TypeError: Can't instantiate abstract class AIPlayer without an implementa
```

And a child that forgets to override the required method fails the same way:

python

```
class BrokenAI(AIPlayer):
    pass

BrokenAI()
```

```
TypeError: Can't instantiate abstract class BrokenAI without an implementa
```

This is a safety net. The mistake is caught the moment you create the object, instead of in the middle of a game. An abstract base class is a **contract**: any class that inherits from it promises to provide these methods. An abstract class can also contain ordinary methods with shared code that its children inherit.

# Applying It: Tic-Tac-Toe AI Players

Here is the shape of the AI family the lab requires. These classes use the `Board` from the last article. Save them in a file called `ai.py`.

python

```
import random
from abc import ABC, abstractmethod


class AIPlayer(ABC):
    """Base class for every computer opponent."""

    @abstractmethod
    def choose_move(self, board, mark):
        """Return a (row, col) tuple for an empty cell on the board."""


class EasyAI(AIPlayer):
    """Picks any empty cell at random."""

    def __init__(self, rng=None):
        if rng is None:
            rng = random.Random()
        self._rng = rng

    def choose_move(self, board, mark):
        return self._rng.choice(board.empty_cells())


class FirstEmptyAI(AIPlayer):
    """Always picks the first empty cell. Predictable, so it is handy for

    def choose_move(self, board, mark):
        return board.empty_cells()[0]
```

The game code that uses them does not know or care which kind it has:

python

```
def take_turn(ai, board, mark):
    row, col = ai.choose_move(board, mark)
    board.place_mark(row, col, mark)
    return row, col
```

python

```
board = Board(3)
take_turn(FirstEmptyAI(), board, "O")      # (0, 0)
take_turn(FirstEmptyAI(), board, "X")      # (0, 1)
print(board)
```

```
O | X |
--+---+--
  |   |
--+---+--
  |   |
```

This is polymorphism at work. `take_turn` calls `choose_move` and trusts the object to answer in its own way. There is no `if level == "easy"` anywhere. To add a Medium or Hard level, you write another class that inherits from `AIPlayer` and fills in `choose_move`. Nothing in `take_turn` changes. The Hard AI will also use recursion from the functions article, in its own helper methods.

Some details worth noticing:

- `isinstance(ai, AIPlayer)` **is** `True` **for every AI.** Code can rely on that to verify it received a valid opponent.
- **The random generator is passed in.** `EasyAI(rng=random.Random(7))` always makes the same choices, because the seed fixes the sequence. This makes tests repeatable, which the pytest article builds on. Without an `rng`, the AI creates its own and plays unpredictably.
- `FirstEmptyAI` **exists for testing.** A fake opponent that is totally predictable lets you test

the game flow without any randomness.

- **Each class has one job.** The base class defines the contract, and each child supplies one strategy.

# Beyond the Basics: Static Typing with Mypy (Optional)

Python is dynamically typed by default. You do not have to declare a variable's type, and Python will not stop you from passing the wrong type into a function until it actually fails at runtime. **Type hints** let you annotate expected types, and **Mypy** is a tool that checks those annotations *before* you run your code, catching type mismatches early.

Mypy is third-party, so install it in your virtual environment first:

bash

```
pip install mypy
```

Here is a simplified version of the bookstore classes, with type hints added:

python

```
class Book:
    def __init__(self, title: str, author: str, price: float) -> None:
        self.title = title
        self.author = author
        self.price = price

    def describe(self) -> str:
        return f"{self.title} by {self.author}"


class EBook(Book):
    def __init__(self, title: str, author: str, price: float, file_size_mb
        super().__init__(title, author, price)
        self.file_size_mb = file_size_mb


def print_description(book: Book) -> None:
```

```
    print(book.describe())


def get_file_size(ebook: EBook) -> float:
    return ebook.file_size_mb
```

Here `book: Book` tells readers and Mypy that this function expects a `Book` object, and `-> None` or `-> float` documents what it returns. Now suppose these two lines appear later in the file:

python

```
get_file_size(Book("Dune", "Frank Herbert", 15.50))
get_file_size(EBook("Dune", "Frank Herbert", 9.99, "big"))
```

Run `mypy` on the file from the command line:

bash

```
mypy your_file.py
```

```
your_file.py:26: error: Argument 1 to "get_file_size" has incompatible typ
your_file.py:27: error: Argument 4 to "EBook" has incompatible type "str";
Found 2 errors in 1 file (checked 1 source file)
```

Your line numbers will differ, because they depend on where you put the two lines in your file. Mypy found both mistakes without running the program. Run the program itself, and the first mistake only appears when `get_file_size` fails with `AttributeError: 'Book' object has no attribute 'file_size_mb'` . Type hints are not enforced by Python, so they only help if a tool such as Mypy (or PyCharm, which flags many of the same problems as you type) checks them.

Across a class hierarchy like `Book` / `EBook` , hints are especially useful. They document exactly which functions expect the base class versus a specific subclass, which helps prevent bugs

where, for example, a plain `Book` gets passed into a function that assumes it has `file_size_mb`. Note that `print_description(ebook)` is fine, because an `EBook` is a `Book`.

# Common Mistakes

| Mistake | What happens | Fix |
|---|---|---|
| Forgetting `super().__init__(...)` in a child's `__init__` | `AttributeError` for the parent's attributes | Call it first, with the parent's arguments |
| Spelling an overridden method differently | The parent's version runs instead | Use the exact same name |
| Overriding with a different return type | Code that expects the parent's behavior breaks | Keep the parent's promises |
| Inheriting when the "is-a" sentence is false | A confusing, fragile design | Use composition (has-a) |
| Deep chains of subclasses | Hard to follow | Keep hierarchies to one or two levels |
| Copying the parent's code into the child | The two versions drift apart | Call `super().method()` and add to it |
| Forgetting `ABC` or `@abstractmethod` | Nothing enforces the contract | Inherit from `ABC` and decorate required methods |
| Creating an object from an abstract class | `TypeError: Can't instantiate abstract class` | Create one of its subclasses |
| A child that does not implement every abstract method | The same `TypeError` | Override all of them |
| Using `isinstance` checks everywhere to choose behavior | Defeats polymorphism | Add the behavior to each class and call the method |
| Assuming type hints are enforced | Wrong types still run | Use Mypy or PyCharm to check them |

# Try It Yourself

Create a file called `inheritance_practice.py` and run it with `python3 inheritance_practice.py`. Try these:

1. Type in the `Book` and `EBook` classes. Create one of each and print `describe()` for both.
2. Call `isinstance` and `issubclass` with different combinations. Which ones are `True`? Which are `False`, and why?
3. Print `EBook.__mro__`. In what order does Python look for a method?
4. Remove the `super().__init__(...)` call from `EBook`, run the code, and read the error. Put it back.
5. Override `sell_copy()` in `EBook`, then sell three copies of a paperback with a quantity of two and three eBooks. Compare the results.
6. Change `describe()` in `EBook` so it replaces the parent's version completely. Then change it to extend the parent's version with `super().describe()`. Which is shorter to maintain?
7. Add an `Audiobook` class that inherits from `Book` and has a `duration_minutes` attribute. Add it to the catalog and run the loop. Did the loop need any changes?
8. Create a class `Audiobook2` that does *not* inherit from `Book` but has a `describe()` method. Put it in the catalog list. Does the loop still work? What is this called?
9. Write an abstract class `Shape` with an abstract method `area()`. Write `Rectangle` and `Circle` subclasses. Put one of each in a list and print every area with one loop.
10. Try to create a `Shape()` directly and read the error. Then write a subclass that forgets `area()` and read that error.
11. Type in the `AIPlayer`, `EasyAI`, and `FirstEmptyAI` classes with the `Board` from the last article. Play a few turns with `take_turn`.
12. Create two `EasyAI` objects with `rng=random.Random(7)` and show that they pick the same cell on the same board.

13. Add a `RandomCornerAI` that picks a random empty corner if one exists, and otherwise any empty cell. Does `take_turn` need to change?

14. Add a `name` attribute and a `describe()` method to `AIPlayer` (a normal method, not abstract) that every subclass inherits. Override it in one child with `super()`.

15. Install Mypy in your virtual environment, add type hints to your `Book` and `EBook` classes, and make Mypy catch one mistake you introduce on purpose.

## Why This Matters for an FDE

Client codebases frequently use inheritance to model families of related things: types of users, types of transactions, types of accounts. Understanding a class hierarchy, what is shared at the parent level versus overridden at the child level, is often necessary just to make a change safely without breaking behavior elsewhere in that hierarchy. Abstract base classes show up whenever a team wants a plug-in design, such as several data sources, several payment providers, or several AI models behind one interface. The agent frameworks you will use later in this program rely on exactly this idea. Static typing tools like Mypy are increasingly standard in production codebases specifically because they catch these mismatches before code ever reaches a client environment.

## Comprehension Check

1. What does `super().__init__(...)` do, and why is it useful in a child class? What happens if you forget it?

2. What is the difference between overriding a method and extending it with `super()`? When would you choose each?

3. Using the checkout screen example, explain why polymorphism reduces the work needed to add a new item type like audiobooks later.

4. Give an example of when you'd want to override a parent class's method rather than just add a new one.

5. `isinstance(ebook, Book)` is `True` but `isinstance(paperback, EBook)` is `False`. Why?

6. What does `@abstractmethod` do, and what happens if you try to create an object from a class that has an unimplemented abstract method?

7. In the Tic-Tac-Toe code, why does `take_turn` not need to change when you add a new kind of AI?
8. A `Game` class needs a `Board`. Should `Game` inherit from `Board`? Explain with the "is-a" test.
9. Why does passing a `random.Random` object into `EasyAI` make it easier to test?
10. What does Mypy check, and does Python itself enforce type hints?

---

← Previous: OOP: Classes, Objects & Encapsulation

Next: TDD with pytest →

## 0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!